

AI Report

We are moderately confident this text is

AI Generated

AI Probability

76%

This number is the probability that the document is AI generated, not a percentage of AI text in the document.

Plagiarism



The plagiarism scan was not run for this document. Go to gptzero.me to check for plagiarism.

tugas praktikum 1 sistem operasi.pdf - 10/19/2025

TUGAS PRAKTIKUM 1 SISTEM OPERASI MUHAMMAD MAULANA NUR SULAIMAN 2410651033 UNIVERSITAS MUHAMMADIYAH JEMBER FAKULTAS TEKNIK PROGRAM STUDI TEKNIK INFORMATIKA 2025 1 DAFTAR ISI Contents Tugas 1: Process Manager.....2
Tugas 2: File Monitor.....9 Tugas 3: Custom Shell.....12 Tugas 4: System Information Tools.....17

2 Tugas 1: Process Manager Konsep dan struktur yang ditunjukkan 1. fork() – membuat proses anak (child process) 2. pipe() – komunikasi antar process (ipc) satu arah 3. shmget(), shmat(), (Shared Memory) – komunikasi antar memori bersama 4. waitpid() – parent menunggu setiap child selesai agar tidak terjadi zombie proses 5. signal handling (sigint) – menangani sinyal Ctrl+C agar program berhenti dengan rapi 6. gettimeofday() – mengukur waktu eksekusi tiap proses anak Alur kerja program 1. Program meminta digit terakhir Jika ganjil, Child 2 akan berkomunikasi dengan parent melalui shared memory 2. Parent meminta tiga input Angka untuk faktorial (Child 1) Batas untuk bilangan prima (Child 2) String untuk dibalik (Child 3) 3. Parent membuat 3 pipe dan 3 child (melalui fork()) Child 1: menghitung faktorial – kirim hasil lewat pipe Child 2: mencari bilangan prima Child 3: membalik string input – kirim hasil lewat pipe 4. Parent menunggu tiap child selesai (waitpid), mencatat waktu eksekusi, lalu: Membaca hasil dari pipe atau shared memory Menampilkan hasil dan durasi eksekusinya 5. Setelah semua selesai, parent menghapus shared memory (jika ada) dan keluar dengan pesan akhir 3 4 5 6 7 8 9

Tugas 2: File Monitor Konsep dan struktur yang ditunjukkan 1. inotify_init(), inotify_add_watch(), dan read() – digunakan untuk memantau perubahan file/direktori 2. open() dan write() – digunakan untuk mencatat aktivitas ke file log.txt 3. signal(SIGTERM, handler) – untuk menangani penghentian program secara aman 4. time() dan localtime – mencatat waktu terjadinya event ke log Alur kerja program 1. User memasukkan digit NIM terakhir. Jika ganjil - setiap perubahan file disertai pesan "notifikasi email dummy" di terminal. 2. Program menjalankan inotify_init() – Membuat instance inotify untuk memantau direktori yang diberikan di argument (argv[1]). 3. Menambahkan watch dengan inotify_add_watch() – Memonitor event IN_CREATE, IN_DELETE, dan IN_MODIFY. 4. Setiap kali ada perubahan di direktori: Program membaca event inotify_fd menggunakan read() Mencatat ke file log (log.txt) dengan timestamp dan PID process. 5. Menangani SIGTERM (misalnya saat dimatikan dengan kill) Menutup semua file descriptor (inotify_fd, log_fd). Keluar secara aman dari program 10 11 12

Tugas 3: Custom Shell Konsep dan Struktur yang ditunjukkan 1. main() – loop uta-

ma shell (while(1)), membaca input pengguna 2. parse_input() – memecah input menjadi token per kata 3. execute_single() – eksekusi command tanpa pipe 4. execute_single_pipe() – eksekusi 1 pipe 5. execute_multi_pipe() – eksekusi beberapa pipe 6. handle_redirection() – menangani >, >>, dan < 7. execvp() – mengganti proses anak dengan program baru 8. dup2() – mengganti input/output standar. Alur program 1. Menjalankan perintah dasar Menggunakan fork() dan execvp() untuk mengeksekusi command dari user 2. Built-in Commands cd <dir> - ganti direktori pwd – tampilkan direktori sekarang echo <teks> - cetak teks exit – keluar dari shell 13 3. Background Process Jalankan perintah di background dengan menambah & diakhir Contoh: sleep 5 &. 4. Redirection (I/O) > - output ke file (overwrite) >> - output ke file (append) < - input dari file Contoh: cat < input.txt > output.txt 5. Piping (|) Menghubungkan output dari satu command ke input command lain NIM ganjil: bisa multiple pipe (cmd1 " cmd2 " cmd3".....) 14 15 16 17 Tugas 4: System Information Tools Konsep dan struktur yang ditunjukkan 1. read_file() – Membaca isi file menjadi string (pakai open() dan read()) 2. trim() – Menghapus spasi di kiri/ kanan string 3. hex_to_ipv4() – mengubah alamat IP dari format hex ke format decimal biasa 4. tcp_state() – Mengubah kode status TCP (hex) menjadi teks seperti LISTEN atau ESTABLISHED 5. show_cpu_info() – Menampilkan model, jumlah core, dan frekuensi CPU 6. show_mem_info() – Menampilkan total, free, dan used memory. 7. show_disk_info() – Menampilkan informasi kapasitas disk 8. show_processes() – Menampilkan daftar PID, nama process, dan status 9. show_net_dev() – Menampilkan nama interface jaringan dan nama jumlah data Rx/Tx 10. show_proc_net_tcp() – khusus ganjil, Menampilkan koneksi TCP aktif 11. main() – Menjalankan semua fungsi di atas secara berurutan Alur Program 1. CPU info (/proc/cpuinfo) Menampilkan model CPU, jumlah core, dan frekuensi (Mhz). Menggunakan open() dan read() untuk membaca isi file 2. Memory info (/proc/meminfo) Menampilkan total, free, cached, buffer, dan memori yang digunakan 3. Disk info (statvfs("/")) Mengambil total, used dan free space pada root filesystem(/) Ditampilkan dalam satuan GB 4. Proses yang sedang berjalan (/proc/<PID>/) Menelusuri direktori / proc untuk mencari folder bernama angka (PID) 5. Network interface Menampilkan setiap interface jaringan - Bytes diterima (Rx) - Bytes dikirim (Tx) 18 6. Network sockets (NIM ganjil) - /proc/net/tcp Menampilkan daftar koneksi TCP aktif di system Mengkonversi alamat hexadecimal ke format IP:port 19 20 21 22 23

FAQs

What is GPTZero?

GPTZero is the leading AI detector for checking whether a document was written by a large language model such as ChatGPT. GPTZero detects AI on sentence, paragraph, and document level. Our model was trained on a large, diverse corpus of human-written and AI-generated text with support for English, Spanish, French, German, and other languages. To date, GPTZero has served over 10 million users around the world, and works with over 100 organizations in education, hiring, publishing, legal, and more.

When should I use GPTZero?

Our users have seen the use of AI-generated text proliferate into education, certification, hiring and recruitment, social writing platforms, disinformation, and beyond. We've created GPTZero as a tool to highlight the possible use of AI in writing text. In particular, we focus on classifying AI use in prose. Overall, our classifier is intended to be used to flag situations in which a conversation can be started (for example, between educators and students) to drive further inquiry and spread awareness of the risks of using AI in written work.

Does GPTZero only detect ChatGPT outputs?

No, GPTZero works robustly across a range of AI language models, including but not limited to ChatGPT, GPT-5, GPT-4, GPT-3, Gemini, Claude, and AI services based on those models.

What are the limitations of the classifier?

The nature of AI-generated content is changing constantly. As such, these results should not be used to punish students. We recommend educators to use our behind-the-scenes [Writing Reports](#) as part of a holistic assessment of student work. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Instead, we recommend educators take approaches that give students the opportunity to demonstrate their understanding in a controlled environment and craft assignments that cannot be solved with AI. Our classifier is not trained to identify AI-generated text after it has been heavily modified after generation (although we estimate this is a minority of the uses for AI-generation at the moment). Currently, our classifier can sometimes flag other machine-generated or highly procedural text as AI-generated, and as such, should be used on more descriptive portions of text.

I'm an educator who has found AI-generated text by my students. What do I do?

Firstly, at GPTZero, we don't believe that any AI detector is perfect. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Nonetheless, we recommend that educators can do the following when they get a positive detection: Ask students to demonstrate their understanding in a controlled environment, whether that is through an in-person assessment, or through an editor that can track their edit history (for instance, using our [Writing Reports](#) through Google Docs). Check out our list of [several recommendations](#) on types of assignments that are difficult to solve with AI.

Ask the student if they can produce artifacts of their writing process, whether it is drafts, revision histories, or brainstorming notes. For example, if the editor they used to write the text has an edit history (such as Google Docs), and it was typed out with several edits over a reasonable period of time, it is likely the student work is authentic. You can use GPTZero's Writing Reports to replay the student's writing process, and view signals that indicate the authenticity of the work.

See if there is a history of AI-generated text in the student's work. We recommend looking for a long-term pattern of AI use, as opposed to a single instance, in order to determine whether the student is using AI.